

Medallion Architecture

Vijayananda J

28th Jul

29th Jul

30th Jul

5th Aug

6th Aug

7th Aug, 2025

Agenda – Day1

- Medallion Architecture
- Why?
- What?
- Parquet

What is Medallion Architecture ?

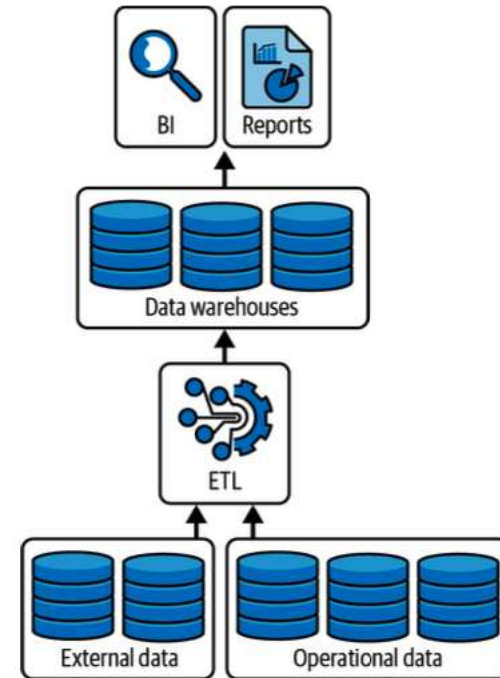
- established name for the pattern: the **Medallion Architecture** (sometimes called the Bronze-Silver-Gold or layered architecture) pioneered in the Delta Lake ecosystem.

Need

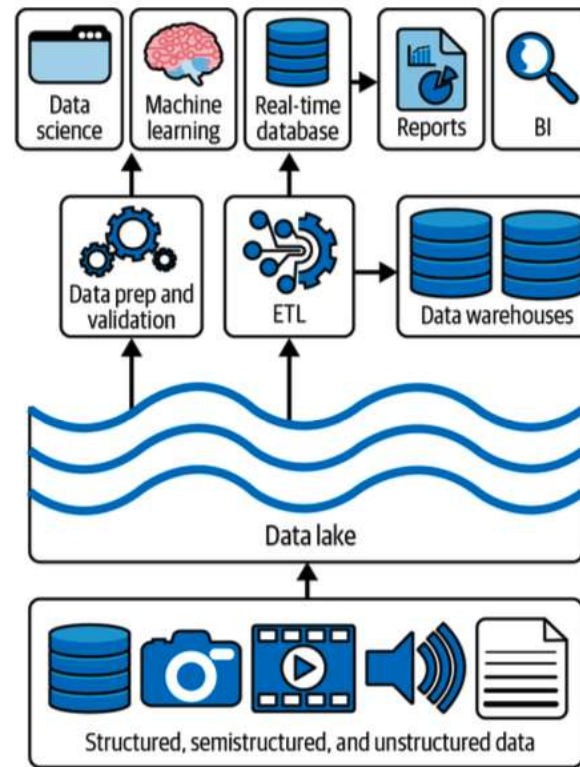
- The Need for Medallion Architecture
- Building the Bronze, silver and Gold layer
- But Why do I need to build these layers?
 - To address problems we are facing in the classic Architecture

Problems in the classical data lake

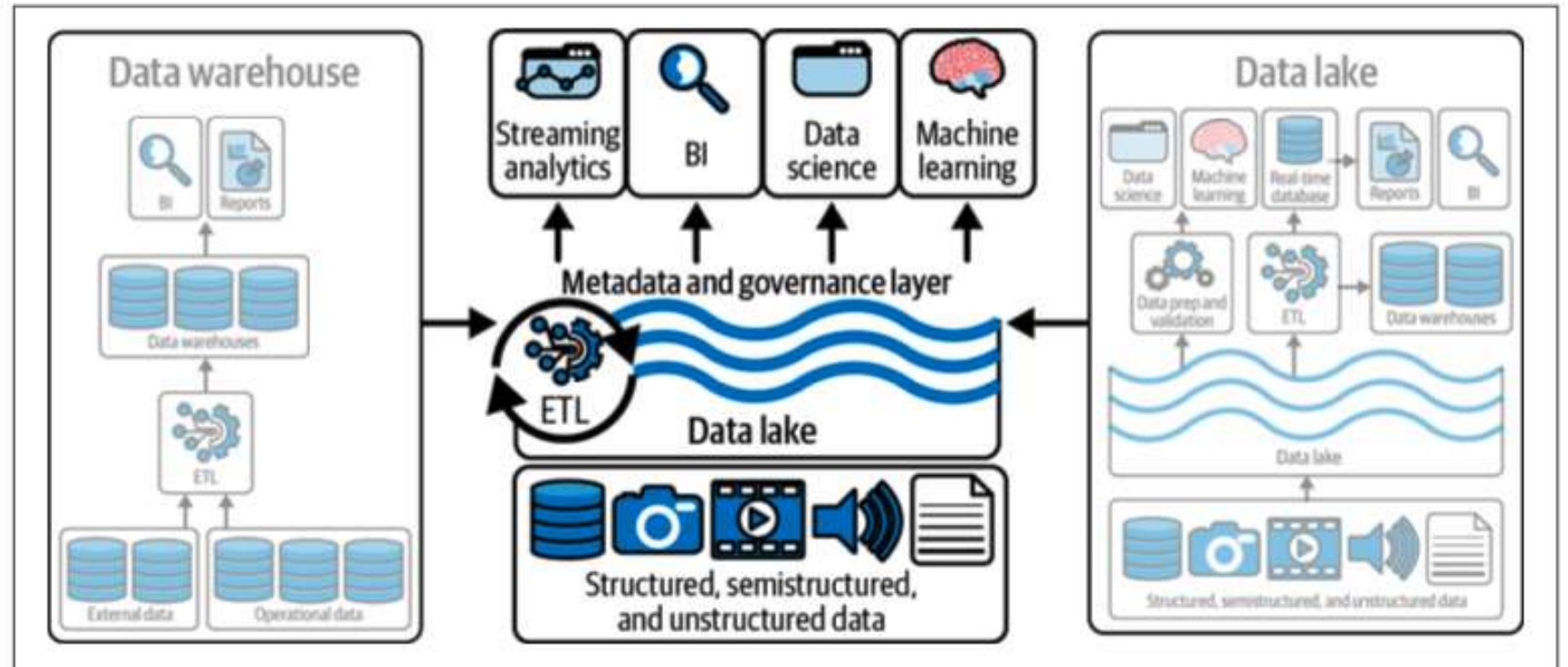
- Data warehousing
- Data lakes



DataLake



Lakehouse

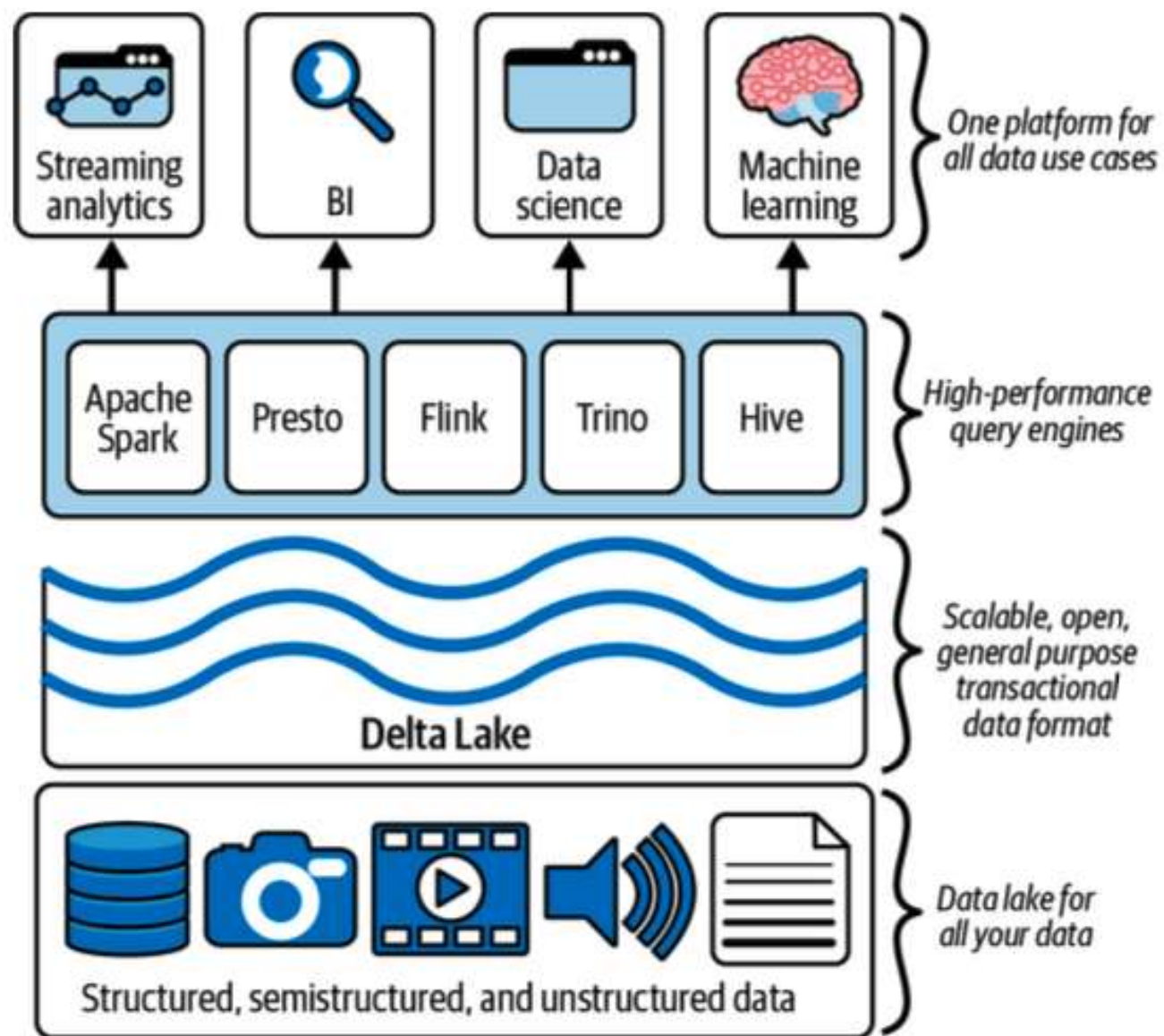


Delta Lake, Apache Iceberg, and Apache Hudi are the most popular open source lakehouse formats.

The name “Delta Lake”

- Project Tahoe
- Delta

Delta lake



Common use cases

- Modernizing data lakes
- Data warehousing
- ML/Data Science
- Streaming Data processing
- Data Engineering
- BI

Key features of Delta Lake

- ACID transactions
- Meta data – Scalable
- Time Travel
- Unified Batch/Streaming
- Schema evolution
- Audit
- Open source
- Performance
- Ease of use

Parquet

Topic

Context & Motivation• Why raw CSV/JSON “just won’t do” at scale• Data lake + Lakehouse needs

Columnar Storage• How Parquet lays out data by column vs row• Benefits for IO & CPU

Compression & Encoding• Snappy vs Gzip vs ZSTD• Dictionary & Delta encoding

Row-Groups & Metadata• What is a row-group?• Min/max statistics in the footer

Partitioning + Directory Pruning• Partition your Parquet on a key• Folder-level skipping

Predicate Pushdown• How engines “peek” at metadata to skip row-groups/files

Schema Evolution• Adding/removing columns, mergeSchema trade-offs

Recap & Q&A

Day 2

- Delta Tables
- ACID
- Unified Batch and Stream
- Schema enforcement

Day 3

- Recap of some of the concepts we discussed y'day
- New concepts in delta lake

ACID quiz

- **Q1.** Why is **ACID compliance critical** for the Lakehouse model, even when using append-only workloads?(new data is only added, and existing data is never modified or deleted.)
-
- **A)** It improves network performance under concurrent writes
- **B)** It simplifies data modeling by avoiding schema enforcement
- **C)** It ensures consistent reads for downstream consumers like BI tools
- **D)** It makes the storage system BASE-compliant for batch workloads

ACID quiz

- Q2. What does **Delta Lake** do that fundamentally makes it **more aligned with ACID** than BASE?
- **A)** Allows schema inference during query execution
- **B)** Supports optimistic concurrency and versioned commits
- **C)** Stores data in object storage that supports eventual consistency
- **D)** Always uses streaming sources for ingestion

Unified Batch and Stream - Quiz

- **Q1.**The term “unified batch and streaming” in Spark refers primarily to:
- A) Using the same compute cluster for both batch and streaming jobs
- B) Ability to execute SQL queries over stream and batch data in the same notebook
- C) Treating both finite (batch) and infinite (streaming) data as unbounded DataFrames with the same semantics
- D) Real-time replication of batch jobs into streaming DAGs behind the scenes

Schema Enforcement & Evolution

Problem in Raw Data Lake

1. Schema drift: Parquet files might have inconsistent schemas across batches.

e.g., Some files: (id: Long, val: String),
Others: (id: Long, val: String, new_col: Double)

2. No central schema registry: There's no governing metadata layer to track changes across writes.

This causes downstream jobs to break or return NULLs when unexpected schema changes occur.

3. Difficult schema evolution: Updating schema requires rewriting all files manually in Parquet.

Delta Lake's Solution

Schema enforcement: Delta rejects writes that don't match the declared schema, preventing silent corruption.

Schema evolution: Enable mergeSchema (or allowSchemaEvolution) to let Delta safely record schema changes in its transaction log.

Delta auto-merges old and new files:

- Older rows show NULL for newly added columns
- New files include the additional column
- Readers always see a unified schema when using .load()

Merge Schema

- When to use
- **Ad hoc or one-off** schema changes in an otherwise stable pipeline.
- You only want to accept new columns on *this* particular write.
- All other writes still strictly enforce the original schema.

```
# You have an existing Delta table at 'path'
with schema: (id LONG, val STRING)

# Build a DataFrame with an extra column 'new_col'
df2 = spark.range(10, 15) \
    .withColumn("val", lit("gamma")) \
    .withColumn("new_col", lit(3.14))

# This append *without* mergeSchema will fail:
df2.write.format("delta").mode("append").save(path)
#           ^ no mergeSchema → schema mismatch error
```

MergeSchema

```
# This append *with* mergeSchema will succeed and register  
the new column |  
df2.write \  
    .format("delta") \  
    .mode("append") \  
    .option("mergeSchema", "true") \  
    .save(path)
```

- When to use
 - Only that one write is allowed to “patch in” the new_col field.
 - You see exactly which commits used mergeSchema in your Delta history.

AutoMerge

```
#Option A)
#Table-level TBLPROPERTIES (Databricks or open-source 2.4+)
#Turn on auto-merge on the table itself

ALTER TABLE delta.`/path/to/table`
  SET TBLPROPERTIES (
    'delta.autoMerge.enabled' = 'true'
  );

#Option B)
#Spark session config (cluster-wide)
# In your notebook or cluster init script
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
```

- With either in place, **every** write that introduces new columns will silently merge them. No `.option("mergeSchema","true")` needed.

MergeSchema Vs Auto-merge

Aspect	mergeSchema per-write	table-level auto-merge
Control	Very explicit—only merges when you ask.	Fully automatic for all writers.
Auditability	History shows exactly which commit used merge.	History won't distinguish auto-merges.
Convenience	A bit more boilerplate per write.	Zero boilerplate—ideal for evolving feeds.
Risk	Lower—schema changes are deliberate.	Higher—any stray column in your DF will get picked up.

If you never
want to
merge...

- If you prefer to create a new table or fail immediately when there's a schema mismatch, instead of merging schemas:
 - **Don't** set mergeSchema or auto-merge.
 - Let the write error bubble up so your pipeline can catch and handle it.
 - Or redirect those writes to a **landing** table where you inspect and evolve the schema manually.

Delta Lake Schema Evolution & Merging Quiz

Q1.

You run the following command on a Delta table with schema (id LONG, name STRING)

```
df = spark.createDataFrame([(1, "Alice", "NY")], ["id", "name", "city"])
df.write.format("delta").mode("append").save("/delta/table")
```

What will happen?

- A) The write will succeed by adding city as a new column.
- B) The write will fail with a schema mismatch error.
- C) The write will succeed, but the city column will be dropped silently.
- D) The write will overwrite the existing schema completely.

Delta Lake Schema Evolution & Merging Quiz

Q2.

Which of the following statements is true when `spark.databricks.delta.schema.autoMerge.enabled` is set to "true" ?

- A) Schema merging will happen automatically for **all** write operations.
- B) Schema merging is enabled for all operations except CREATE TABLE.
- C) Schema merging is still controlled by the specific write method (e.g., DataFrameWriter vs SQL).
- D) It disables schema enforcement completely.

Delta Lake Schema Evolution & Merging Quiz

Q3

You've enabled `mergeSchema=true` in a notebook, but the write still fails due to schema mismatch. Which of the following could be the cause?

- A) The cluster has Table ACLs enabled.
- B) The Delta table is partitioned.
- C) You used SQL `INSERT INTO` instead of DataFrame API.
- D) Both A and C.

Time Travel & Versioning

- **Every Delta commit gets a monotonically increasing version number** and an atomic JSON file in `_delta_log/`. Because of that, you can reliably reconstruct exactly what your table looked like at **any** of those versions—or at any wall-clock timestamp between them.
- **Use cases:**
 - **Reproduce historical reports:** rerun last quarter's dashboard against the exact data you had then.
 - **Audit & compliance:** inspect who changed what, when.
 - **Debug late-arriving data:** roll back to version N, fix the bad file, then re-apply.

Time Travel and versioning

APIs:

- By version:

```
spark.read  
  .format("delta")  
  .option("versionAsOf", 1)  
  .load(path)
```

- By timestamp:

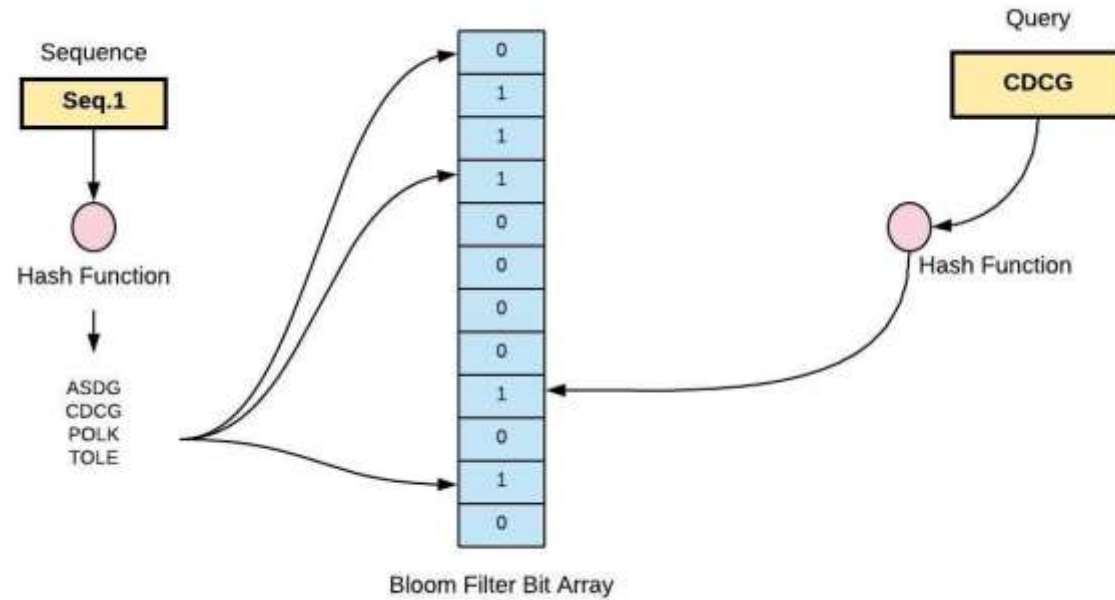
```
spark.read  
  .format("delta")  
  .option("timestampAsOf", "2025-07-28T10:47:55.000Z")  
  .load(path)
```

- History: shows who committed each version, when, and what operation they ran."

```
DESCRIBE HISTORY delta.`/path/to/table`
```

Bloom filters

(Burton Howard
Bloom)



- Bloom filters are probabilistic data structures that efficiently test for set membership. They excel at quickly determining if an element might be in a set.
- This makes them useful when memory is a concern

False positives and false negatives

- False positive - You **predicted something is true**, but in reality, it is **false**. (False was positive)
 - **False Positive:** Test says X has cancer, but X actually doesn't → **Unnecessary stress, treatment**
- False negative - You **predicted something is false**, but in reality, it is **true**. (False was negative)
 - **False Negative:** Test says X is healthy, but X actually has cancer → **Dangerous! Missed treatment**
- False negatives are more dangerous than False positives
- General rule of thumb –
 - If **missing the event** leads to harm, **false negatives** are more dangerous
 - If **raising a false alarm** causes cost or distrust, **false positives** may matter more

Are bloom filters err on FP or FN?

- Bloom filters – we want speed in fetching data.
- So when bloom filter is used to check for a presence of a data in a parquet file, its ok if Bloom filters respond, I don't know and then we read the parquet and find out.
- But when it says 'No', the data better be not present in the parquet file.
 - So when IsPresent(value) ? Returns No , it should be 100% true
 - If is returns yes/Maybe, it is ok not to be 100% true
 - Since our goal is to skip reading the files as much as possible.
 - So BF should be 100% accurate for FN (i.e No FN)

Case	Bloom Filter Accuracy
Element is actually not in the set	Always says "not present" →  no false negatives
Element is in the set	Might say "present" even when it isn't →  false positives possible

Bloom filters in Delta lake

- **Delta Lake** uses **Bloom filters** at the **file level** to **speed up queries** by skipping files that **definitely don't contain** the searched values.
- This is helpful especially when:
 - You're filtering on a **high-cardinality column** (like user IDs or invoice numbers)
 - You want to reduce I/O on large datasets

Create Bloom filter

```
from pyspark.sql import SparkSession
import random

spark = SparkSession.builder.getOrCreate()

# Sample data with high-cardinality column `user_id`
data = [(i, f"user_{random.randint(1, 10_000)}") for i in range(1, 1_000_001)]
df = spark.createDataFrame(data, ["id", "user_id"])

# Save as Delta with Bloom filter on `user_id`
df.write.format("delta") \
    .option("delta.dataSkippingNumIndexedCols", "1") \
    .option("delta.dataSkippingIndexedColNames", "user_id") \
    .option("delta.dataSkippingBloomFilterEnabled", "true") \
    .option("delta.dataSkippingBloomFilterIndexMaxNumItems", "1000000") \
    .mode("overwrite") \
    .save("/tmp/delta_bloom_example")
```

```
df_bloom = spark.read.format("delta").load("/tmp/delta_bloom_example")

# Query with value that may or may not exist
filtered = df_bloom.filter("user_id = 'user_1234'")

filtered.explain(True) # Shows physical plan with file skipping
filtered.show()
```

What does a developer do?

- Is the developer responsible for using the Bloom filter at query time?
 - **No.** Once configured during **write**, Bloom filters in Delta Lake are **automatically used by the Delta engine** during queries — **if the query has a filter condition** on the indexed column.

Step	Who's Responsible
Enable Bloom filters during write	✓ Developer
Use a filter in the query (WHERE col = 'val')	✓ Developer
Call or reference the Bloom filter manually	✗ Not needed
Trigger file skipping manually	✗ Not needed

```
df.write.format("delta") \
  .option("delta.dataSkippingBloomFilterEnabled", "true") \
  .option("delta.dataSkippingIndexedColNames", "user_id") \
  .save("/path/to/table")
```

What the Delta Engine Does Automatically ?

Once Bloom filters are enabled and the table is written with indexed columns, Delta automatically:

- Reads the query filter predicate (e.g., WHERE user_id = 'user_1234')
- Checks if there's a **Bloom filter index** on that column
- Consults the Bloom filters for each file
- **Skips any files** where the Bloom filter says, "this value is definitely not here"

Skipping data and faster access

- **Bloom filters** (if enabled)
- **Partition pruning** (folder paths)
- **Parquet footer row-group stats** (if deeper pushdown is needed)
- **Delta-log stats skipping** (JSON “stats” field in the `_delta_log` `addFile` entries)

File-skipping hierarchy in Delta Lake

Stage	What's Checked	Source of Stats
1. Partition Pruning	Does the query filter on a partition column?	Folder path names (/.../country=US/)
2. Delta-Log File-Level Stats	Does the file's min/max/null counts overlap filter?	JSON in _delta_log "add" entries
3. Bloom Filters (<i>opt-in</i>)	Does the Bloom filter indicate no matching rows?	.bloomFilterStats in Parquet footer
4. Parquet Row-Group Stats	Which row-groups' min/max overlap the predicate?	Parquet file footers
5. Actual Data Scan	Read the remaining row-groups and apply full filter.	On-disk Parquet column chunks

Check your
understanding

Scenario	Best Mechanism
customer_id = 'C123'	Bloom Filter, Min-Max Stats, Partition Pruning
age BETWEEN 30 AND 40	Bloom Filter, Min-Max Stats, Partition Pruning
product_name LIKE 'Sony%'	Bloom Filter, Min-Max Stats, Partition Pruning
country = 'US' (and partitioned)	Bloom Filter, Min-Max Stats, Partition Pruning
email IN (...)	Bloom Filter, Min-Max Stats, Partition Pruning

Bloom filters Quiz

- Which of the following conditions must be **true** for a Bloom filter in Delta Lake to **actually reduce query scan time**?
-
- **A)** The column has low cardinality with repeated values
- **B)** The query contains a filter on the column with a Bloom index
- **C)** The query uses a full table scan and ignores filter conditions
- **D)** The Bloom filter was created using ALTER TABLE ... ADD BLOOM FILTER INDEX

Day 4-

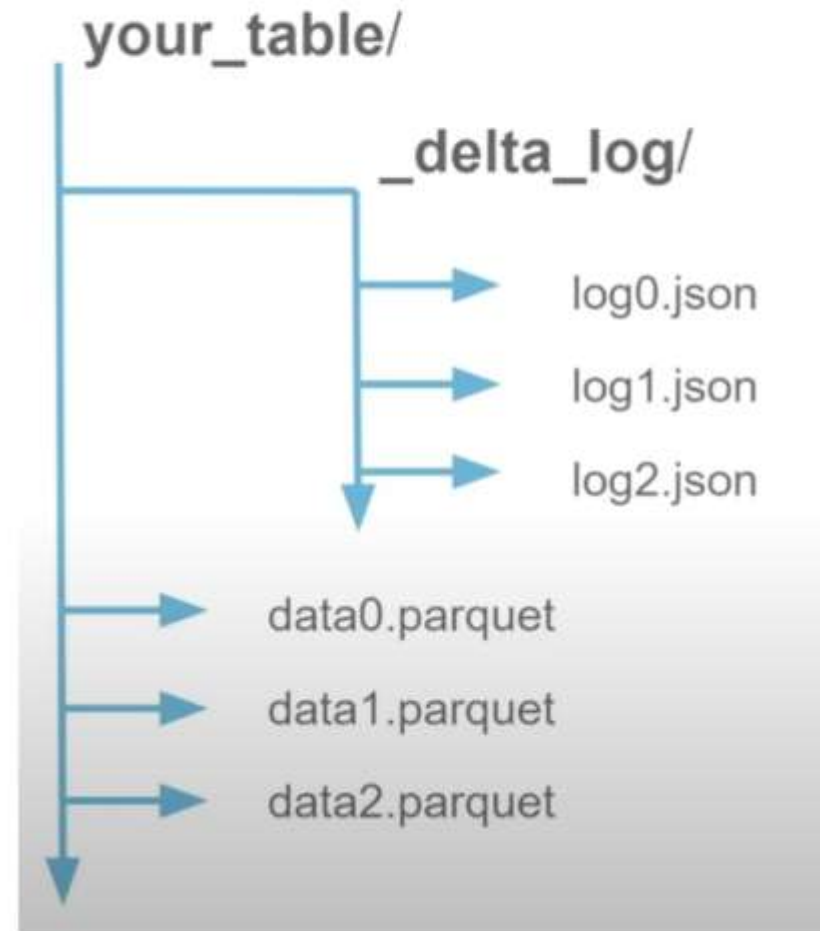
- Topics to cover
 - File storage structure
 - Meta data management
 - Medallion Architecture

Meta data Management

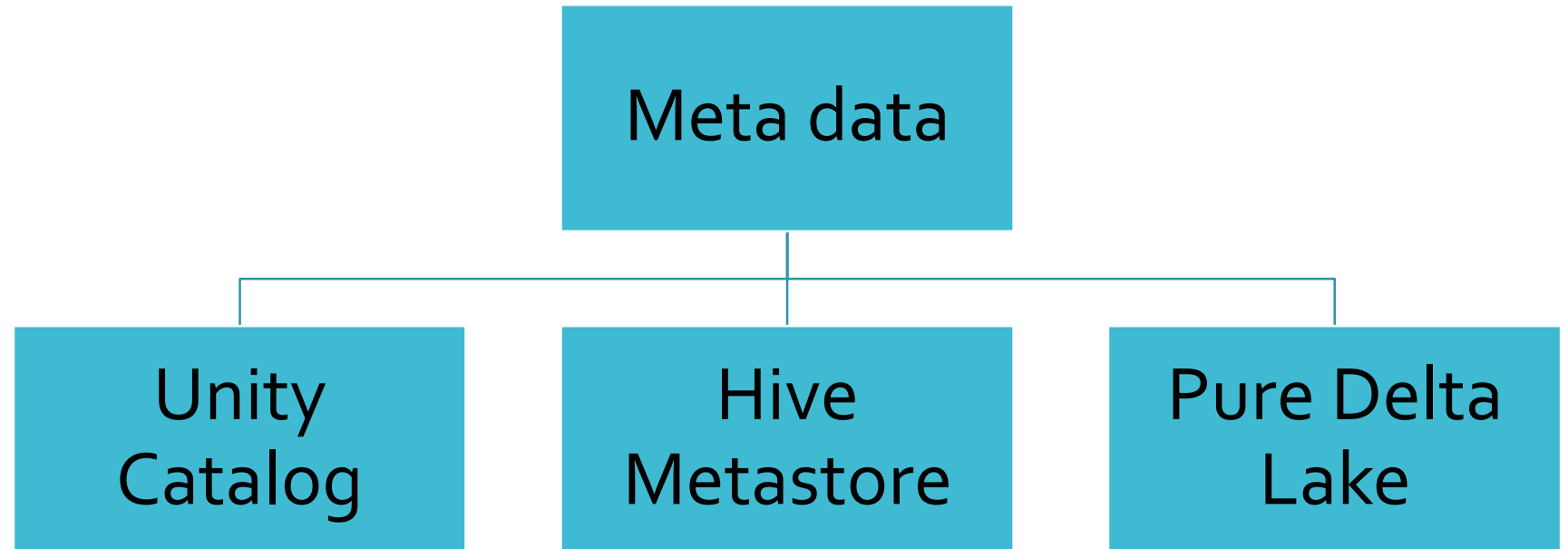
- **Data discovery & governance**
 - Enables data consumers to find and trust the right tables.
 - Drives lineage, access control, auditing.
- **Performance & optimization**
 - Partition- and file-level statistics let the engine prune data and choose efficient plans.
- **Schema evolution & enforcement**
 - Tracks column additions, type changes, defaults.
- **Auditability & reproducibility**
 - History of all writes/updates supports debugging and compliance.

File storage structure

File Storage



Meta data
mgmt



Comparison

Category	Unity Catalog	Hive Metastore	Pure Delta Lake
Structural	Table definitions, cols	Same, plus partition map	Schema in _delta_log JSON
Operational	Access policies, lineage	GRANT entries	Author/timestamp in commits
Physical	Location URLs	Location URLs	File add/remove list

Best Practices & Recommendations

- **Always register production tables**
 - Important for governance.
- **Collect statistics**
 - ANALYZE TABLE ... COMPUTE STATISTICS FOR COLUMNS ...
- **Automate schema evolution**
 - Enable mergeSchema in pipelines and ensure metastore sync.
- **Hive Meta store**
 - For external tables
- **Leverage UC lineage**
 - Surface end-to-end data flows to stakeholders.
- **Fallback on pure Delta**
 - For prototypes, but plan to onboard into UC/HMS before “go-live.”

Unity catalog meta data

```
Unity Catalog (multi-workspace control plane)
└─ Catalogs/
    └─ finance_catalog/
        └─ Schemas/
            └─ raw/
                └─ Tables/
                    └─ transactions ← external Delta
                        └─ Columns: id, user_id, amount, ts...
                    └─ customers
                        └─ Columns: cust_id, name, region...
            └─ silver/
                └─ Tables/
                    └─ transactions_clean ← managed Delta
                        └─ Columns: id, user_id, amount, category...
            └─ gold/
                └─ Tables/
                    └─ daily_summary
                        └─ Columns: date, total_amount, avg_amount...
        └─ ACLs & Policies/ ← column- and row-level grants
    └─ marketing_catalog/
        └─ Schemas/
            └─ bronze/
            └─ silver/
            └─ gold/
        └─ ACLs & Policies/
```

Hive Meta Store meta data

```
Hive Metastore (`spark_catalog`-per-workspace)
└─ Databases/
    └─ default/
        └─ Tables/
            └─ bronze_covid_nyt    ← external Delta on DBFS
                └─ Columns: date, state, cases, deaths...
            └─ silver_covid_nyt    ← external Delta on DBFS
            └─ gold_covid_summary
        └─ Grants/                ← database- and table-level only
    └─ analytics_db/
        └─ Tables/
        └─ Grants/
    └─ reporting_db/
        └─ Tables/
        └─ Grants/
```

Recommendation Workspaces in Production - UC

```
/Workspaces
├── Dev/
│   ├── Repos/                ← feature branches, notebooks under /Repos/dev/...
│   ├── Clusters/             ← all-purpose compute for exploration
│   ├── Jobs/                 ← ad-hoc notebook jobs
│   ├── UnityCatalog (dev_uc) ← UC metastore for Dev
│   │   ├── Schemas/
│   │   │   ├── dev_myteam.bronze/
│   │   │   │   └── Tables: raw_events, raw_customers...
│   │   │   ├── dev_myteam.silver/
│   │   │   │   └── Tables: clean_events, customer_dim...
│   │   │   └── dev_myteam.gold/
│   │   │       └── Tables: event_summary, churn_model_input...
│   │   └── ACLs/              ← dev-only permissions
│   ├── Staging/
│   │   ├── Repos/            ← merged PRs, release candidates
│   │   ├── Clusters/          ← serverless/UAT-sized clusters
│   │   ├── Jobs/              ← smoke tests & UAT pipelines
│   │   ├── UnityCatalog (stage_uc) ← UC for Staging
│   │   │   ├── Schemas/
│   │   │   │   ├── stage_myteam.bronze/
│   │   │   │   ├── stage_myteam.silver/
│   │   │   │   └── stage_myteam.gold/
│   │   │   └── ACLs/          ← tighter than Dev, open to QA
│   └── Prod/
│       ├── Repos/             ← tagged release branches
│       ├── Clusters/           ← locked, auto-terminate, infra-managed
│       ├── Jobs/               ← scheduled Medallion pipelines
│       ├── UnityCatalog (prod_uc) ← UC for Production
│       │   ├── Schemas/
│       │   │   ├── prod_myteam.bronze/
│       │   │   │   └── Tables: raw_events...
│       │   │   ├── prod_myteam.silver/
│       │   │   │   └── Tables: clean_events...
│       │   │   └── prod_myteam.gold/
│       │   │       └── Tables: event_summary...
│       │   └── ACLs/           ← strict RBAC & row-level security
```

Recommendation Workspaces in Production - HMS

```
/Workspaces
├── Dev/
│   ├── Repos/                ← your feature branches & notebooks
│   ├── Clusters/             ← all-purpose compute for exploration
│   ├── Jobs/                 ← ad-hoc notebook jobs
│   └── spark_catalog (Hive Metastore)
│       ├── Databases/
│       │   ├── dev_myteam_bronze/
│       │   │   └── Tables:
│       │   │       ├── raw_events
│       │   │       └── raw_customers
│       │   ├── dev_myteam_silver/
│       │   │   └── Tables:
│       │   │       ├── clean_events
│       │   │       └── customer_dim
│       │   └── dev_myteam_gold/
│       │       └── Tables:
│       │           ├── event_summary
│       │           └── customer_ltv
│       └── Grants/            ← DB- and table-level privileges for Dev users
├── Staging/
│   ├── Repos/                ← merged PRs, release candidates
│   ├── Clusters/             ← QA/UAT-sized clusters
│   ├── Jobs/                 ← smoke tests & UAT pipelines
│   └── spark_catalog (Hive Metastore)
│       ├── Databases/
│       │   ├── stage_myteam_bronze/
│       │   │   └── Tables: raw_events, raw_customers
│       │   ├── stage_myteam_silver/
│       │   │   └── Tables: clean_events, customer_dim
│       │   └── stage_myteam_gold/
│       │       └── Tables: event_summary, customer_ltv
│       └── Grants/            ← tighter DB- and table-level grants for QA
└── Prod/
    ├── Repos/                ← tagged "release" branches only
    ├── Clusters/             ← locked, autoscale, infra-managed
    ├── Jobs/                 ← scheduled Medallion pipelines
    └── spark_catalog (Hive Metastore)
        ├── Databases/
        │   ├── prod_myteam_bronze/
        │   │   └── Tables: raw_events, raw_customers
        │   ├── prod_myteam_silver/
        │   │   └── Tables: clean_events, customer_dim
        │   └── prod_myteam_gold/
        │       └── Tables: event_summary, customer_ltv
        └── Grants/            ← strict production-only privileges
```

Real world folder Structure

```
/dev/...  
/stage/...  
/prod/...
```

```
/prod/  
  retail/  
    orders/  
      bronze/  
        raw_orders/  
        kafka_ingest/  
      silver/  
        cleaned_orders/  
        enriched_orders/  
      gold/  
        order_summary/  
        regional_kpis/  
    customers/  
      bronze/  
      silver/  
      gold/  
  finance/  
    invoices/  
    payments/  
  
/dev/  
  ... same tree structure ...
```

Two patterns in production

```
Unity Catalog: mycompany_uc
├─ Schema: dev_finance
├─ Schema: dev_sales
├─ Schema: stage_finance
├─ Schema: prod_finance
└─ Schema: prod_sales
```

```
Dev Workspace → UC Metastore: mycompany_dev_uc
Stage Workspace → UC Metastore: mycompany_stage_uc
Prod Workspace → UC Metastore: mycompany_prod_uc
```

Problem definition for medallion Architecture

- **Start with the End-User Need**
 - Show me my weekly sales by region, product line, and customer segment.
- In order to satisfy this query, we need to have the following schema in our DB

week_ start	region	product_ line	customer_ segment	total_ orders	total_ revenue	avg_order_ value
2025-07-28	APAC	Electronics	Enterprise	1,234	₹12,345,67	₹10,005
2025-07-28	EMEA	Apparel	SMB	987	₹4,560,123	₹4,621
...	...	...	...	...	...	...

Highly curated: all joins, aggregations, and derivations done

Conformed: same dimensions & metrics across dashboards

Fast: ready for BI tools, data science, ML scoring

This highly curated data is the gold data

How does the source data look like?

- **Event logs** (clickstreams, transactions)
- **Third-party feeds** (CRM exports, partner APIs)
- **Sensor / device dumps** (IoT telemetry)
- **DB snapshots** (full or incremental inserts)

Typically landed “as-is” into a **Bronze** zone

```
/mnt/data/bronze/sales_events/  
├─ 2025-07-28_events_0001.parquet  
├─ 2025-07-28_events_0002.json  
└─ ...
```

Raw to gold

- **Clean**
 - Drop or impute (filling missing values) nulls
 - Normalize date/timestamp formats
 - Filter out test or invalid rows
- **Enrich**
 - Lookup product metadata (e.g. join on product master)
 - Geocode IP addresses to region
 - Tag VIP customers from CRM
- **Transform**
 - Cast types (string → date, decimal → numeric)
 - Derive new columns (week_start, order_value tiers)
 - Aggregate (daily → weekly, raw events → summaries)
- **Conform & Optimize**
 - Ensure consistent column order & naming
 - Partition on query keys
 - Collate small files into larger Parquet/Delta files

What are the options?

- **Monolithic ETL job**
 - One gigantic script: ingest → clean → enrich → publish
 - Slow iterations, brittle code, hard to debug
- **ELT in a data warehouse**
 - Load everything into Snowflake/Redshift, then SQL transforms
 - High storage/compute costs, limited streaming support
- **Micro-services or event streams**
 - Each service publishes cleansed events downstream
 - Complex service orchestration, tight coupling
- **Custom in-app analytics**
 - Each application writes directly to its own reporting tables
 - Data silos, inconsistent definitions across apps

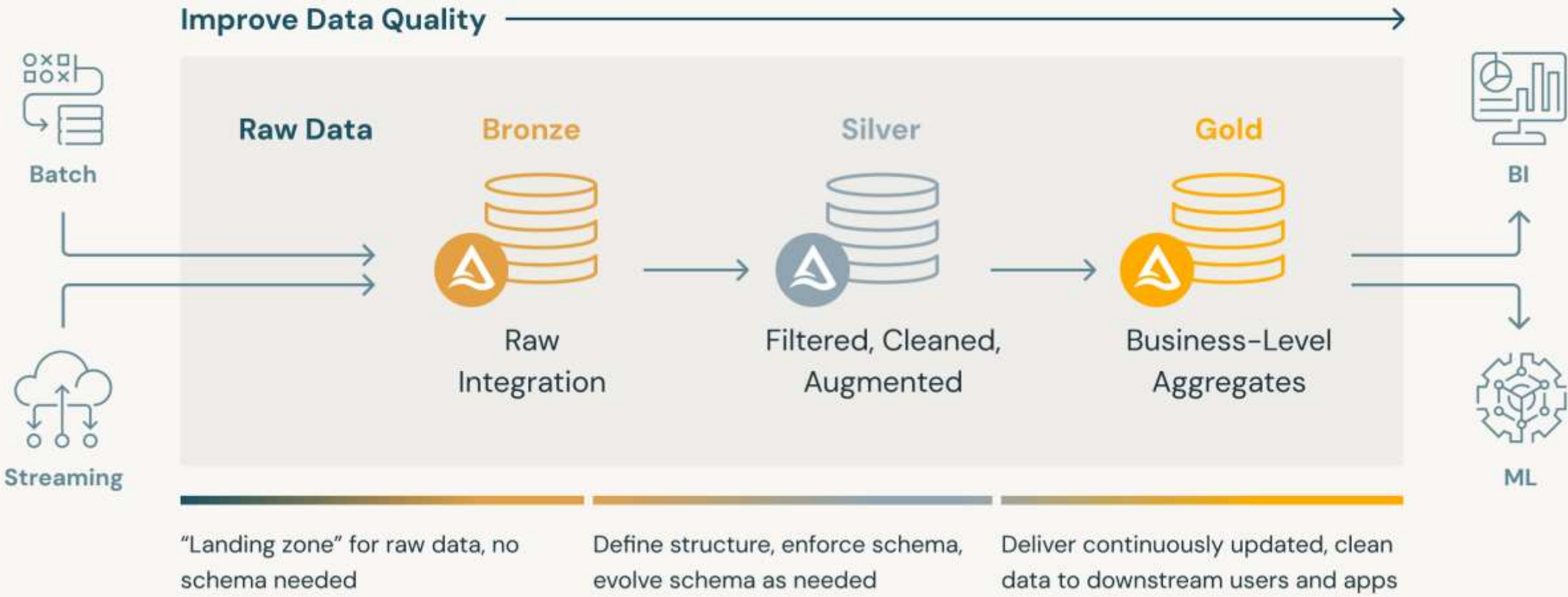
None of these give us both:

Incremental, reliable pipelines

Clear separation of concerns (landing vs. cleaning vs. serving)

Easy reprocessing when logic changes

Building reliable, performant data pipelines with DELTA LAKE



Comparison of strategies

Aspect	Monolithic ETL	ELT in Data Warehouse	Micro-services / Event Streams	In-app Analytics	Medallion (Bronze/Silver/Gold)
Separation of Concerns	✗ Everything in one pipeline; brittle	⚠ Staging vs. analytics tables, but logic still centralized in SQL scripts	⚠ Each service owns its slice, but cross-cutting logic scattered	✗ Reporting logic embedded in each app	✓ Clear layers: raw landing → cleansing/quality → analytics-ready
Re-processability	✗ Hard to re-run just part of the flow	⚠ Can reload staging, but warehouse transforms often destructive	✗ Difficult to rewind streams across services	✗ No single source of truth	✓ Bronze retains all raw data; you can rebuild Silver/Gold at any time
Streaming Support	⚠ Possible with custom code, but ad-hoc	✗ Pure batch focus	✓ Native streaming, but no consistent layering	✗ Typically batch exports	✓ Both batch & streaming ingested into Bronze; same logic applied downstream
Cost & Compute Efficiency	⚠ Single large jobs, often over-provisioned	⚠ Warehouse storage + compute can be expensive for raw data	⚠ Many small services; overhead per service	⚠ Duplicate compute in each app	✓ Bronze minimizes compute on raw ingest; Silver/Gold compute scaled to need; storage cheap for Bronze raw data
Governance & Security	⚠ One big job, limited audit trails	⚠ Warehouse offers roles/privileges, but raw data often unmanaged	⚠ Service-specific auth; inconsistent auditing	✗ No central governance	✓ Unity Catalog/Hive for Silver/Gold; raw audit via Delta logs; fine-grained ACLs at each layer
Observability & Lineage	✗ Logs scattered in ETL tool	⚠ Warehouse lineage features, but raw source lost	⚠ Difficult to track cross-service flows	✗ Hard to trace across apps	✓ delta "DESCRIBE HISTORY" at each layer; UC lineage captures table-to-table flows
Ease of Evolution	✗ Massive refactors on logic change	⚠ SQL scripts brittle at scale	⚠ Orchestration choreography gets complex	✗ Each app must be updated separately	✓ Change logic in Silver only; rebuild downstream without touching Bronze or Gold definitions

Bronze Layer

- **Bronze layer (raw data)**
- The **Bronze layer** - *land all the data* from external source systems. The table structures in this layer corresponds to the source system table structures "as-is," along with any additional metadata columns that capture the load date/time, process ID, etc.
- The focus in this layer is –
 - Change Data Capture
 - Change Data Feed
 - ability to provide a historical archive of source (cold storage),
 - data lineage, auditability,
 - reprocessing if needed without rereading the data from the source system.

Bronze layer

```
bronze = bronze.withColumn("ingest_ts", current_timestamp()) \
               .withColumn("process_id", lit("trial_batch"))
```

- **Lineage & Auditing**
 - ▶ ☐ You can trace back “when did row X arrive?”
- **Latency Monitoring**
 - ▶ ☐ Compare event_timestamp (source time) vs ingest_ts to measure pipeline lag
- **Reprocessing & Backfills**
 - ▶ ☐ If you re-run a load, new rows get a new ingest_ts, so you know which run they came from
- **Debugging**
 - ▶ ☐ Filter “all rows that ingested after 10 PM” to isolate a suspect load

Why process_id?

- In practice, it helps in
- **Debugging:** Quickly filter WHERE process_id = 'daily_telemetry_ingest|12345_67890'
- **Auditing:** Know exactly which job run produced each row
- **Reprocessing:** Rerun a failed pipeline and compare the new process_id

Process ID

- In a real-world pipeline `process_id` uniquely identifies **which exact run** (and ideally which job) wrote each batch of data.
- Helps in trace, debug, and re-process precisely.
- best practices:
 - Most schedulers or orchestration systems assign a unique run identifier to each pipeline execution.
 - Include a Human-Readable Prefix

```
pipeline_name = "daily_telemetry_ingest"  
process_id = f"{pipeline_name}|{job_id}_{run_id}"  
# e.g. "daily_telemetry_ingest|12345_67890"
```

Process_Id

- Fallback to Timestamps (when you have no orchestrator)

```
from pyspark.sql.functions import date_format, current_timestamp
# in your DataFrame:
.withColumn("process_id", date_format(current_timestamp(), "yyyyMMdd_HH:mm:ss"))
```

- Result: 20250731_143522 — unique to the second.

```
.withColumn("job_run_id", lit(process_id)) \
.withColumn("ingest_ts", current_timestamp())
```

- job_run_id ties to your orchestrator
- ingest_ts tells you exactly when the rows landed

Silver Layer

- In the **Silver layer** of the lakehouse, the data from the Bronze layer is matched, merged, conformed and cleansed ("just-enough") so that the Silver layer can provide an "Enterprise view" of all its key business entities, concepts and transactions.
- The Silver layer
 - brings the data from different sources into an Enterprise view
 - enables self-service analytics for ad-hoc reporting, advanced analytics and ML.
 - serves as a source for Departmental Analysts, Data Engineers and Data Scientists to further create projects and analysis to answer business problems via enterprise and departmental data projects in the Gold Layer
 - Does not contain project-specific complex transformations and business rules are applied while loading the data from the Silver to Gold layer.

Activities in silver layer

- **Filter out bad rows**
 - Drop any records missing required key fields (e.g. WHERE date IS NOT NULL).
- **Type-cast & rename**
 - Convert raw strings into proper types (STRING → DATE/INT/DOUBLE) and give them business-friendly names.
- **Drop unused columns**
 - Remove any raw fields you don't need downstream (e.g. internal flags, debug payloads).
- **Flatten nested structs**
 - Turn a column of type STRUCT<...> into top-level columns (device.os, device.region → os, region).

Activities in silver layer ...

- **Normalize column names**
 - Standardize casing and punctuation (e.g. eventTimestamp → event_timestamp).
- **Trim & clean strings**
 - Strip whitespace, collapse multiple spaces, enforce lowercase/uppercase where appropriate.
- **Drop duplicates**
 - .dropDuplicates([<natural_key_columns>]) so that retries or late arrives don't bloat the table.
- **Surrogate key generation**
 - Create a unique key (e.g. hash(user_id, event_timestamp, event_type)).

Activities in silver layer ...

- **Derive new columns**
 - Compute helpful baked-in fields (e.g. `event_hour = hour(event_timestamp)`).
- **Conformance join**
 - Left-join to a small “feature lookup” or “dimension” table so every record carries a business code (e.g. `event_type`→`feature_category`).
- **Mask or obfuscate PII**
 - Hash or redact sensitive fields according to your security policy.

Activities in silver layer ...

- **Fill defaults / NULL handling**
 - Replace missing numeric fields with 0 or sentinel values where business logic requires.
- **Partition / repartition**
 - Repartition your DataFrame on appropriate keys (e.g. event_date) before writing for high-performance reads.
- **File-size tuning**
 - Coalesce or OPTIMIZE ... ZORDER to ensure files are in the 100–500 MB sweet spot.
- **Quality assertions / expectations**
 - In DLT or Spark assert that “cases >= 0” or “region IN (valid_region_list)” and either drop or quarantine bad rows.

Gold layer...

- In the Gold layer the cleansed, conformed Silver tables and build analytics-ready, “project-specific” tables—often in a star schema (fact + dimensions)
- or highly de-normalized wide tables optimized for reporting.
- Apply final business logic, aggregations, and any last data-quality rules, and register these tables in your metastore (HMS or UC) for BI/SQL users.

Typical Gold-Layer Transform Steps

- **Define your fact table**
 - Aggregate events into metrics (e.g. daily counts, sums, averages).
- **Build dimensions**
 - Extract slowly changing or descriptive attributes (e.g. date dim, region dim, user dim).
- **Join fact → dimensions**
 - Produce a wide, denormalized star model for fast BI queries.
- **Apply final data-quality checks**
 - Ensure no null keys, enforce referential integrity.
- **Partition & optimize**
 - Partition fact by date; run OPTIMIZE / Z-ORDER on high-cardinality keys.

Day 5

- Recap of Bronze, Silver, Gold layers
- Z ordering
- Incremental Processing in Medallion
- Limitations of Medallion
- Delta Live Tables

Z ordering

- **Concept of Z-Ordering**
 - Why plain partitioning isn't always enough?
 - **Partitioning** (e.g. by date) groups data into folders so Spark only scans relevant partitions.
 - But within each partition, files may still contain rows spanning a wide range of values for other columns you filter on (e.g. user_id, region).

Z -order

- What is Z-Ordering?
 - A **space-filling curve** technique that maps multidimensional data (e.g. (x, y, z)) into a one-dimensional sequence while preserving “locality.”
 - Points that are close in multidimensional space end up near each other in the 1D ordering.
 - When applied to data files, it **clusters** rows with similar values on one or more columns into the same file or row-group blocks.

Applicability of Z order in delta lake

- **Data skipping & file pruning**
 - Delta's transaction log stores **file-level statistics** (min/max for each column).
 - When you run a query with filters, Spark examines each file's stats to see if it can skip reading that file entirely.
- **OPTIMIZE ... ZORDER BY**
 - **OPTIMIZE table ZORDER BY (col1, col2, ...)** triggers two steps:
 - **Compaction**: merges many small files into larger ones (e.g. 512 MB target).
 - **Clustering**: physically reorders rows within those files along the Z-curve of the specified columns.
 - **Result**: files are now **clustered** by your most-common filter predicates, making file-skipping much more effective.

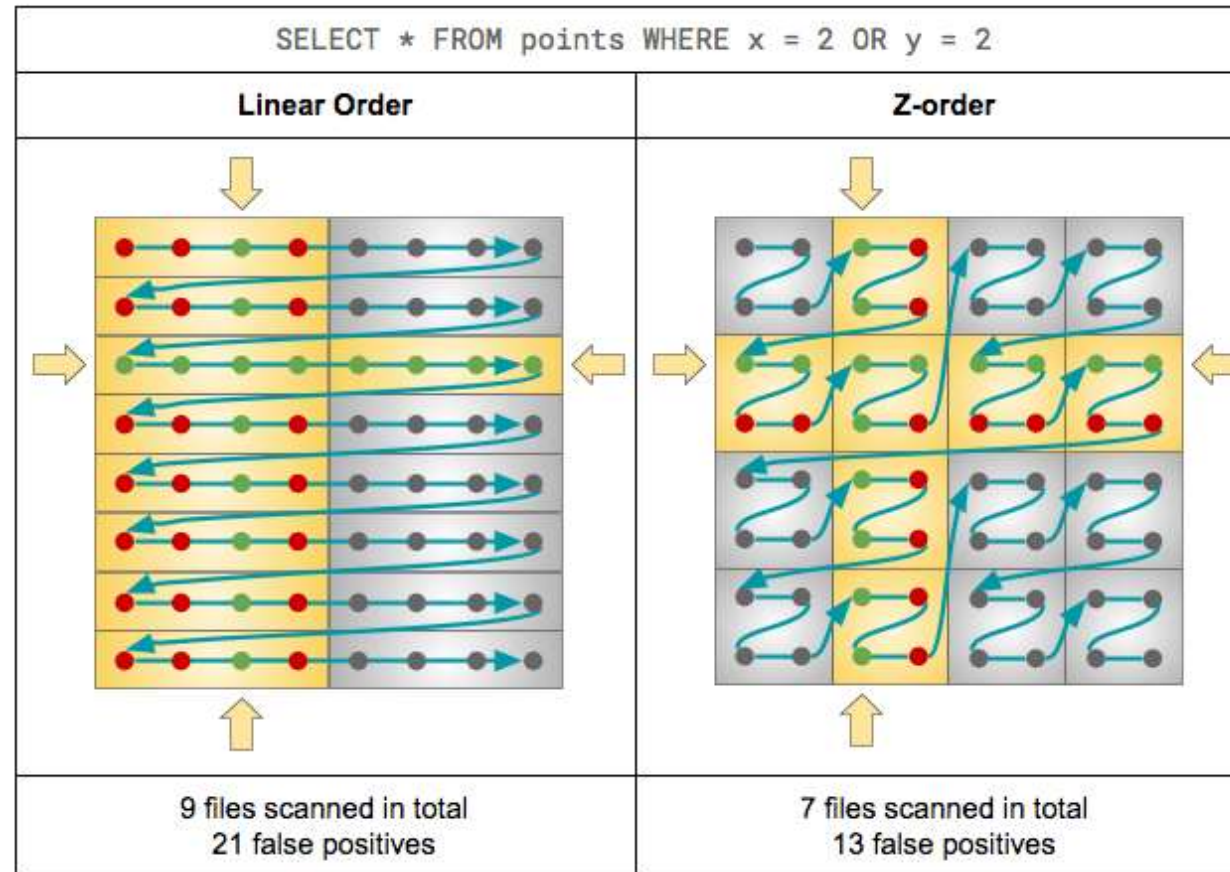
When to use Z-Ordering ?

- You have **high-cardinality** columns (e.g. user ID, order ID, region, device type) that you frequently filter on.
- Your Silver or Gold tables still scan many files even after partition pruning.
- You want **one-off** or **periodic** compactions to improve read performance, without reengineering your upstream writes.

Low cardinality vs high cardinality

- **Low-Cardinality Columns**
 - **Definition:** Only a handful of distinct values (e.g. five regions, a dozen product categories).
- **Behavior with Z-Order:**
 - Z-ordering will cluster those few values tightly. This often ends up with one or two files that contain *only* a given value (e.g. "West"), and the rest contain the others.
 - **Result:** Very effective pruning for filters on that column—Spark can skip almost all non-matching files, cutting I/O dramatically.
- **When to Use:**
 - Frequent filter on that column alone (or with another low-card filter)- near-perfect file skipping
- **High-Cardinality Columns**
 - **Definition:** Thousands or millions of distinct values (e.g. customer IDs, session tokens).
- **Behavior with Z-Order:**
 - Z-ordering on a high-card column still clusters similar values together
 - We still get micro-clusters of adjacent values (e.g. user_1001–user_1024) in each file, but it's unlikely any file holds only your target user.
 - **Result:** May still have to open many files to satisfy a filter on a single user, so file-skip rates improve only modestly (or sometimes negligibly).
- **When to Use:**
 - Combine that filter with other predicates (e.g. date ranges), so Z-order on (user_id, date) gives joint clustering.
 - Z-order to accelerate range-based scans (e.g. user IDs between 1,000 and 2,000) rather than point lookups.

Z-Order example



21 false positives came down to 13

False positives are files that Spark **predicted it needed to read** (because their metadata said they *might* contain x and y = 2 rows) but in fact contained *no* such data.

Originally Spark mis-guessed on **21** files, so it opened them and scanned them unnecessarily.

After Z-ordering, better clustering meant only **13** files still had ambiguous metadata—so Spark had to open just 13 extra files.

how to choose which columns to Z-Order on and how often to run OPTIMIZE in your pipelines.

- **Pick the Right Columns**
 - Query Patterns First: Analyze your most frequent and selective predicates.
 - Low-Cardinality Wins: Single-column Z-Order on a few distinct values yields near-perfect pruning.
 - Multi-Column for High-Cardinality: Combine a high-card column with a date or region to get joint clustering (e.g. (user_id, event_date)).
- **When to Run OPTIMIZE ... ZORDER BY**
 - Initial One-Time Triage: After you onboard a large, cold table—compact & Z-Order once to fix file layout.
 - Periodic Maintenance: Schedule nightly or weekly jobs when table size grows by X% or new partitions arrive.
 - Event-Driven Triggers: Run post-backfill or after major upstream schema changes.
- **Cost vs. Benefit Trade-Off**
 - Compute & I/O Cost: Each OPTIMIZE reads and rewrites entire target partitions.
 - Performance Gain: Balance job frequency against query speedup (measure skip rates & latencies).
 - Monitoring: Use Spark UI or table metrics to track file-skip improvements over time.

Incremental Processing in medallion

- **Incremental Processing**** refers to the idea of
 - only ingesting and transforming the *new or changed* data since the last run—
 - rather than reprocessing your entire dataset end-to-end every time.

** very important feature

Key points

- **Why It Matters:**
 - Performance & Cost: Processes megabytes of churn instead of terabytes of static data.
 - Freshness: Enables sub-hourly or even near-real-time updates to Silver & Gold layers.
 - Reliability: Simplifies retry logic—if a micro-batch fails, you only reprocess that batch.
- **Key Enablers in Delta Lake:**
 - Change Data Feed (CDF): Read inserts, updates, and deletes by table version.
 - MERGE INTO: Idempotent upserts of new/changed rows.
 - Watermarking or File-Listing: DIY approaches for append-only sources.
- **Typical Silver-Layer Flow:**
 - Read last checkpoint (version, timestamp, or file log).
 - Pull only the deltas from Bronze.
 - Clean and transform that delta.
 - MERGE it into the Silver table.
 - Advance the checkpoint.

Incremental Processing in medallion

- **How Delta “snapshots” work vs. record deduplication**
 - **Delta’s transaction log** ensures that each write (whether it’s a new file or a MERGE) produces a *new table version* that clients will see atomically.
 - **But** it does *not* automatically dedupe or collapse multiple rows with the same business key (e.g. UID) when you simply append new data files.
- Say in the bronze layer
- x.parquet contains
 - UID = 123 , value = “foo” and silver layer has run
- Later there is an update for UID = 123, now Y.parquet, has updated information of UID 123
 - UID = 123, value = “bar”
- What should happen at the silver layer?

Incremental Processing in medallion

- This “duplicate-row” problem shows up exactly in your **Silver** layer if merely **append** updated Bronze files without a MERGE (or DELETE) step.
- In an append-only Silver pipeline, every new Bronze record becomes a new row in Silver—even if it has the same business key (UID) as an earlier row.
- That means the Silver table will accumulate “stale” and “fresh” versions side-by-side, and a simple `SELECT *` returns both (foo and bar)

Approach to address this

1. Create a tiny checkpoint table

```
CREATE TABLE IF NOT EXISTS demo.silver_cdf_checkpoint (  
  last_version BIGINT  
) USING DELTA;
```

2. On job start: read that single number

```
ckpt = spark.table("demo.silver_cdf_checkpoint").collect()  
last_version = ckpt[0]["last_version"] if ckpt else 0
```

3. Pull CDF from last_version+1 to current

```
changes = (spark.read  
  .format("delta")  
  .option("readChangeData", "true")  
  .option("startingVersion", last_version+1)  
  .option("endingVersion", current_version)  
  .table("demo.bronze_events"))
```

Approach to address this...

3. Pull CDF from last_version+1 to current

```
changes = (spark.read
    .format("delta")
    .option("readChangeData", "true")
    .option("startingVersion", last_version+1)
    .option("endingVersion", current_version)
    .table("demo.bronze_events"))
```

4. After your MERGE, overwrite the checkpoint with current_version

```
spark.createDataFrame([(current_version,)], ["last_version"]) \
    .write.mode("overwrite") \
    .saveAsTable("demo.silver_cdf_checkpoint")
```

Exercise

- Add Checkpoint table for your exercise

Exercise – Can
we have our
own DIY
without
relying on the
“merge into”
feature/CDF?

Delta Live Tables

- Delta Live Tables is a framework on Databricks that simplifies the development and operation of data pipelines by managing the full lifecycle of your ETL/ELT workflows declaratively.
- Instead of writing Spark jobs, define “**live tables**” using SQL or Python;
- DLT handles pipeline orchestration, dependency resolution, and operational best practices for you.
- **Key characteristics:**
 - **Declarative pipeline definitions:** You declare what the tables should contain, not how to compute them step by step.
 - **Managed infrastructure:** Databricks automatically provisions and scales compute resources for your pipeline.
 - **Continuous or triggered execution:** Pipelines can run on a schedule or continuously process data as it arrives.
 - **Built-in monitoring & quality:** Data quality rules, expectations, and health metrics are first-class constructs.

Benefits of Using Delta Live Tables

Benefit	Description
Declarative & Readable	Pipelines are defined as simple SQL views or Python functions with decorators—easy to understand and maintain.
Automated Orchestration	DLT figures out task ordering, handles retries on failure, and ensures recovery from checkpoints.
Built-in Quality Checks	Native support for expectations lets you enforce schema, null-checks, uniqueness, and custom rules.
Scalability & Cost Efficiency	Auto-scaling of clusters and resource isolation per pipeline prevents over-provisioning or interference.
Versioning & Reproducibility	Pipelines are versioned; you can roll back to previous definitions and replay history with full audit trails.
Seamless Integration	Works hand-in-glove with Unity Catalog, Databricks SQL, and Delta Lake features like CDF and time travel.

What about Checkpointing ?

- **Automatic Change Detection**
DLT auto-enables CDF on Bronze tables and pulls only new/updated/deleted rows each tick.
- **Internal Checkpoints**
Maintains its own Delta-based state so you never write or manage a checkpoint table.
- **Declarative Upserts**
In streaming mode, DLT generates the necessary MERGE INTO logic for you—no manual code.

What that essentially means?

```
@dlt.table(  
    comment="Raw events with CDF",  
    table_properties={"delta.enableChangeDataFeed": "true"}  
)  
def bronze_events(): ...
```

DLT does

- Enable CDF on that table

- Track versions internally

- Feed only the delta records into downstream tables

- Persist its own checkpoint so you don't see or manage it

Exercise

- Migrate to the DLT on your exercise

Medallion – when does it add value?

Dimension	“Low” (Simplified ELT)	“High” (Medallion Pays Off)
Data volume per run	< 1 GB	> 10 GB
Schema width & churn	≤ 10 columns, very stable (changes < 1/month)	> 20 columns, evolving schema (≥ 1 change/week)
Transform complexity	1–2 light transformations (filter, cast)	multiple joins, windowed aggregations, UDFs
Data freshness / velocity	pure batch, hourly or daily	sub-hourly, micro-batches, even low-latency streaming
Duplication & quality	upstream guaranteed unique, little de-duplication required	retries/late arrivals, need dropDuplicates & audit trails
Number of downstream users	1–2 analysts, ad-hoc notebooks	> 5–10 teams or dashboards (each with different semantic needs)
Conformance / lookup joins	zero or one small lookup table	many conformance/lookups (app→category, user→segment, geo joins)

Limitations of Medallion

- **Operational Overhead**
Three layers $\Rightarrow$ three pipelines to build, test, monitor, and document.
- **Latency Impact**
Each Bronze $\rightarrow$ Silver $\rightarrow$ Gold hop adds I/O, serialization, and checkpoint delays.
- **Over-engineering Small Workloads**
For tiny, once-a-day jobs, multiple layers add unnecessary complexity.
- **Tool-chain Lock-In**
Tightly coupled to Spark/Delta; harder to support other engines like Snowflake or BigQuery.
- **Team Skillset Mismatch**
Requires data-engineering expertise; small teams may prefer simpler ETL.

Alternative architecture and patterns

Alternative	What It Is	When to Use It
One-step Delta	Read raw data → <code>.write.format("delta")</code> → final table.	Very small volume or single transformation; you just need a reliable data store with ACID.
Lambda Architecture	Separate batch & speed layers, then merge.	Extremely low-latency must-have plus solid batch; but beware complexity of two code paths.
Kappa Architecture	Pure streaming, materialize views at needed grain.	If your entire workload is streaming and you can compute aggregates on the fly (no batch fall-backs).
Data Mesh	Domain-oriented, self-serve “data products.”	Large enterprises with autonomous teams—each domain owns its own data pipeline and publishes “products,” avoiding central bottlenecks.

One-step Delta

```
# 1) Read raw data (could be JSON, CSV, Parquet, etc.)
df = spark.read.json("/path/to/raw/events/*.json")

# 2) (Optional) Do your "T" in ETL right here:
#     filter, cast, derive new columns, join to lookups, etc.
clean_df = (
    df
    .filter("event_type IS NOT NULL")
    .withColumn("ingest_ts", current_timestamp())
    .withColumn("event_date", to_date("ts"))
)

# 3) Write *directly* to your final Delta table
clean_df.write
    .format("delta")
    .mode("overwrite")          # or "append" for incremental
    .save("/path/to/delta/events")
```

Under the hood, Delta Lake gives :

ACID transactions (atomic commits, isolation across concurrent jobs)

Time travel via the `_delta_log`

Schema enforcement & evolution if you enable `mergeSchema`

What have we covered so far?

- Datawarehouse vs Datalake Vs Lakehouse
- Parquet features – File storage structures, Automatic row-groups, stats, Partition pruning, Predicate push-downs
- Delta Table – Versioning, ACID, Unified Batch and Stream, Schema Evolution, Time travel, Schema merge, Zorder, Bloom Filters,
- Medallion architecture – Bronze, silver, Gold, Incremental Upserts, Checkpointing, Alternatives, Limitations, Onestep delta

Excercise

COVID lakehouse exercise

File	Description	Granularity	Time span
us-counties.csv	Full historical county-level case & death counts, maintained by the New York Times.	County & date	2020-01-21 through 2021-02-11
us-counties-recent.csv	"Sliding window" of the last 7 days of that same county-level data, for faster nightly loads.	County & date	2021-02-05 through 2021-02-11
us-states.csv	Daily state-level aggregates of cases & deaths, also from the NYT.	State & date	2020-01-21 through 2021-02-11

Schema

Column	Type	Meaning
date	DATE	The calendar date of the report.
county	STRING	County name (only in the first two files).
state	STRING	State name (both county-level and state-level files).
fips	INT	5-digit FIPS code (unique geographic identifier).
cases	INT	Cumulative total of confirmed COVID-19 cases, as of that date.
deaths	INT	Cumulative total of confirmed COVID-19 deaths, as of that date.

Data

1) us-counties.csv (≈1.1 M rows)

- Who & why:** Published daily by the NYT, it contains *every* U.S. county's running totals from January 21, 2020 onward.
- Use case:** Historical backfill, full-snapshot loads in Bronze so you can replay or audit from Day 1.
- Key point:** Because it grows every day, it's large—keep it in Bronze and only use Silver/Gold aggregates downstream.

2) us-counties-recent.csv (≈94 000 rows)

- Who & why:** Same schema, but only the *most recent 7 days* of county counts.
- Use case:** Speeds up nightly incremental loads—rather than scanning the full 1 M+ file, you can land just the last week and merge it in.
- Key point:** union this with the full file on first load, then on future runs only process it (plus any real-time corrections).

3) us-states.csv (≈18 000 rows)

- Who & why:** NYT's state-level daily aggregates—one line per state per date.
- Use case:** Optionally land in Bronze if you want pre-aggregated state summaries; you can skip this if focus is about county-rollups in Silver 2.
- Key point:** Useful for building very fast state-level dashboards without needing to group up 3 000+ counties every time.

Context – use case

You're a data engineer at a state health department tasked with building a reliable analytics pipeline to track COVID-19 case counts. Policy makers need both raw historical data for compliance audits and up-to-the-minute summaries for daily briefings.

- Data Ingestion (Bronze Layer):**

- Batch:** Every night, you land the full historical county-level case counts (us-counties.csv) plus a 7-day “recent” file (us-counties-recent.csv).
- Stream:** Corrections (e.g. late-reported outbreaks, data cleanup) arrive in real time as small JSON files.

- Raw Landing Zone Goals:**

- Store the untouched nightly dumps and stream corrections as Delta tables in your Hive Metastore (“covid” database).
- Tag each record with a source label (counties_full, counties_recent, OR correction/stream) and an ingest_ts so you can trace lineage and handle late arrivals.

- Silver Layer (Clean & Upsert):**

- Silver 1 (County-level):**
 - Merge the historical + recent files, then incrementally pull in stream/correction rows.
 - Deduplicate on (report_date, county) via MERGE (or CDF + readChangeData) so the latest case count always wins.
- Silver 2 (State-daily):**
 - Roll up the cleaned county data into daily state totals for faster downstream access.

- Gold Layer (Fast Aggregates):**

- Gold 1:** Compute each state's 7-day moving average of cases/deaths (for trend analysis).
- Gold 2:** Produce a weekly summary by state (for weekly briefings).

Topics covered in the exercise

- **Delta & Parquet Basics**
 - Reading CSVs into DataFrames and writing them out as Delta.
 - How Delta sits on Parquet (row-group stats, predicate push-down, file pruning).
- **Bronze Layer**
 - Landing batch CSVs (full + recent) and streaming corrections into raw Delta tables.
 - Tagging with source and ingest_ts, registering tables in the Hive Metastore.
- **Incremental Processing & Upserts**
 - DIY vs. true Change Data Feed (CDF) patterns.
 - Using MERGE INTO or CDF + _change_type to do exactly-once upserts into Silver.
- **Silver Layer**
 - **Silver 1:** county-level cleansed fact table (deduplication, schema casts).
 - **Silver 2:** roll-up to state-daily aggregates.
- **Gold Layer**
 - **Gold 1:** 7-day moving average by state (window functions vs. pre-materialization).
 - **Gold 2:** calendar-week summary by state.

Topics covered in the exercise

- **Schema Evolution**
 - How to evolve your Silver/Gold schemas with mergeSchema (adding new fields without rewrites).
- **Layout & Performance Tuning**
- **Partition Granularity:** day vs. month for Silver 2.
- **Z-Ordering** on (report_date, county) for Silver 1.
- **Bloom Filter Indexes** on county for point lookups.
 - (Brief discussion of bucketing if time permits.)
- **Retention & Vacuum** (*optional stretch*)
 - Setting Delta log retention and running VACUUM to manage storage vs. time-travel depth.
- **Checkpointing & Orchestration Patterns**
 - How we track “last processed” via a checkpoint table vs. DLT’s built-in state.
 - Sketch of imperative Spark jobs vs. a Delta Live Tables alternative.